

Infrastructure as Code (IaC):

Writing and managing Kubernetes configurations as code (YAML manifests) can become complex, especially when dealing with large and distributed applications.

Versioning and rollbacks:

Managing versioning and rollbacks of Kubernetes deployments can be challenging, especially when multiple microservices are involved.

Service discovery and load

balancing: Properly managing service discovery and load balancing for microservices can be complex, especially in a dynamic and distributed environment like Kubernetes, OpenShift, ROSA, EKS, AKS and GKE.

Monitoring and observability:

Setting up comprehensive monitoring and observability solutions for Kubernetes clusters and microservices can be daunting, requiring the integration of multiple tools.

Scalability and resource

management: Scaling microservices dynamically while optimising resource allocation requires careful planning and continuous monitoring.

Secret management: Handling sensitive information like API keys, passwords, and certificates securely in Kubernetes can be tricky.

Application configuration:

Managing application configurations across different environments consistently can be challenging, especially when multiple configurations are needed for various microservices.

Continuous delivery pipelines:

Designing efficient and robust CI/CD pipelines for Kubernetes deployments requires a deep understanding of container images, Helm charts, and Kubernetes manifests.

Security and compliance: Ensuring security best practices and compliance standards are met in a containerised environment like Kubernetes can be complex.

Resource cleanup: Cleaning up unused resources and ensuring proper

resource management in Kubernetes clusters can become a challenge, especially when dealing with dynamic scaling and frequent deployments.

DevOps tools

DevOps tools like Terraform, Ansible, Jenkins, AWS CodePipeline, AWS CodeDeploy, and Azure DevOps are widely used and powerful in traditional infrastructure and application management; they might face some challenges when used with Kubernetes. While these tools work well with infrastructure provisioning and configuring cloud based applications, they struggle when used with Kubernetes.

Paradigm mismatch: The DevOps tools mentioned before follow a procedural approach, whereas Kubernetes is declarative. This difference can lead to challenges in mapping workflows and resource management between these tools and Kubernetes.

Resource management:

DevOps tools often have their own resource management mechanisms, which can conflict with Kubernetes resource management.

Kubernetes native concepts: Some of the previously mentioned tools might not fully leverage Kubernetes-native concepts and features, limiting their potential to take advantage of Kubernetes capabilities.

Kubernetes manifests: Kubernetes resources are defined in YAML manifests, which might not be natively supported by some DevOps tools, making it challenging to manage and update Kubernetes configurations.

Secret management: Handling Kubernetes secrets securely is crucial, and integrating this process seamlessly with the above DevOps tools might require additional effort.

Rolling updates and rollbacks:

Kubernetes provides built-in support for rolling updates and rollbacks. Integrating these capabilities effectively with these DevOps tools can be

complex, especially if the DevOps tools are not Kubernetes-aware.

Service mesh integration:

Kubernetes service mesh technologies like Istio or Linkerd might not be well-integrated with these DevOps tools, leading to difficulties in managing advanced networking features.

Monitoring and observability:

Some DevOps tools may not have native integrations with Kubernetes monitoring and observability solutions, making it harder to gain insights into the health and performance of Kubernetes deployments.

Dynamic environment changes:

Kubernetes is designed to handle dynamic environments with ease, but this can pose challenges for some DevOps tools that are more suited to static environments.

Scalability and self-healing:

While Kubernetes excels at dynamic scaling and self-healing, integrating these features effectively with some DevOps tools might require additional configurations and adjustments.

Continuous integration and continuous delivery (CI/CD) pipelines:

These have become a cornerstone of modern software development practices. Among the numerous CI/CD solutions available, Tekton Pipelines stands out as a powerful and flexible platform, enabling teams to automate and streamline their software delivery workflows. We will explore the features, benefits, and applications of Tekton Pipelines, shedding light on how these have revolutionised the software development process.

Tekton

Tekton is a cloud native solution which is an open source project and Kubernetes-native resource for building, testing, and deploying applications. It consists of Tekton Pipelines, which provide the building blocks of supporting components, such as Tekton CLI and Tekton Catalog, that make Tekton a complete ecosystem. Tekton runs inside Kubernetes or its other variants like AWS ROSA.